IS2108 PE

python3 -m venv venv

source ./venv/bin/activate

Install Django:

python -m pip install Django==5.2.17

Copy PIP install to txt:

python -m pip freeze > requirements.txt

Create a new Django project for [Student Management System]with the name “[]”

django-admin startproject []

Move to [], Create a new app for [].
python manage.py startapp []

```
# Application definition

INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
    # add below, if they ask to build module in adminmodul
e, comment other apps first, then finish the model come bac
k uncomment it
    "adminmodule",
    # "studentmodule",
]
```

UML Model

```
from django.db import models


# Create your models here.
class Student(models.Model):

    studentId = models.AutoField(primary_key=True)
    studentNumber = models.CharField(max_length=9, blank=Fa
lse, unique=True)
    fullName = models.CharField(max_length=64, blank=False)


class Course(models.Model):

    courseID = models.AutoField(primary_key=True)
    courseCode = models.CharField(max_length=12, blank=Fals
e, unique=True)
    courseName = models.CharField(max_length=128, blank=Fal
se)

    students = models.ManyToManyField(Student, related_name
="courses_students")


class CourseGrade(models.Model):

    CourseGradeID = models.AutoField(primary_key=True)
    finalLetterGrade = models.CharField(max_length=2, blank
=False)

    student = models.ForeignKey(Student, on_delete=models.R
ESTRICT, blank=False)
    course = models.ForeignKey(Course, on_delete=models.RES
TRICT, blank=False)
```

All primary key values are numerical whole number and should be auto-generated. (models.AutoField())

```
studentId = models.AutoField(primary_key=True)
```

Required (blank=False)

```
courseName = models.CharField(max_length=128, blank=False)
```

Must be Unique (unique=True)

```
courseCode = models.CharField(max_length=12, blank=False, u
nique=True)
```

Can be NULL (null = True)

```
courseName = models.CharField(max_length=128, blank=False,
null=True)
```

Choices of Word (choices = [])

```
    LETTER_GRADE_CHOICES = [
        ('A+', 'A+'),
        ('A', 'A'),
        ('A-', 'A-'),
        ('B+', 'B+'),
        ('B', 'B'),
        ('C+', 'C+'),
        ('C', 'C'),
        ('D+', 'D+'),
        ('D', 'D'),
        ('E', 'E'),
        ('F', 'F'),
    ]

    finalLetterGrade = models.CharField(max_length=2, blank
=False, choices=LETTER_GRADE_CHOICES)
```

Many-to-one (ForeignKey())

```
student = models.ForeignKey(Student, on_delete=models.RESTRICT, blank=False, related_name='student_coursegrades')
```

Many-to-many (ManyToManyField())

```
students = models.ManyToManyField(Student, related_name="courses_students")
```

One-to-one (OneToOneField())

```
student = models.OneToOneField(Student, on_delete=models.CASCADE, related_name='address')
```

- **CASCADE:** “If the parent goes, the children go too.”
- **RESTRICT:** “You can’t delete the parent while it still has children.”

```
class Classroom(models.Model):
    name = models.CharField(max_length=50)

class Student(models.Model):
    name = models.CharField(max_length=50)
    classroom = models.ForeignKey(Classroom, on_delete=models.RESTRICT)

class Homework(models.Model):
    title = models.CharField(max_length=100)
    student = models.ForeignKey(Student, on_delete=models.CASCADE)
```

CASCADE (Student → Homework)

If you delete the student “Ali”, **all of Ali’s homework is deleted too**. Homework with no student is useless, so removing it automatically makes sense.

RESTRICT (Classroom → Student)

If you try to delete “Class 3A” while students are still in it, **Django stops you with an error**. You don’t want to lose students just because a classroom was removed. Move them to another class first, then delete the classroom.

Create the database table:

python manage.py makemigrations
python manage.py migrate

LOAD DATA in app level

Setup app level: urls.py in project level:

```
"""
URL configuration for studentmgtsys project.

The `urlpatterns` list routes URLs to views. For more infor
mation please see:
    https://docs.djangoproject.com/en/5.2/topics/http/urls/
Examples:
Function views
    1. Add an import:  from my_app import views
    2. Add a URL to urlpatterns:  path('', views.home, name
='home')
Class-based views
    1. Add an import:  from other_app.views import Home
    2. Add a URL to urlpatterns:  path('', Home.as_view(),
name='home')
Including another URLconf
    1. Import the include() function: from django.urls impo
rt include, path
    2. Add a URL to urlpatterns:  path('blog/', include('bl
og.urls'))
"""

from django.contrib import admin
from django.urls import include, path

from . import views

urlpatterns = [
    path("admin/", admin.site.urls),
    # ensure all is add in
```

```
    path("adminmodule/", include("adminmodule.urls")),
    path("", views.index, name="studentmgtsys_index"),
]
```

Add views.py in project lv:

```
from django.shortcuts import render


def index(request):

    return render(request, "index.html")
```

Create a folder templates, then add index.html

Go to setting.py, setup the base directory:

```
TEMPLATES = [
    {
        "BACKEND": "django.template.backends.django.DjangoT
emplates",
        "DIRS": [BASE_DIR / "studentmgtsys/templates"],
        "APP_DIRS": True,
        "OPTIONS": {
            "context_processors": [
                "django.template.context_processors.reques
t",
                "django.contrib.auth.context_processors.aut
h",
                "django.contrib.messages.context_processor
s.messages",
            ],
        },
    },
]
```

Go to app level [adminmodule]:

In views.py in app lv:

```
from django.http import HttpResponse

# add the model that need to insert
from .models import Course


# Create your views here.
def loadData(request):

    try:

        c = Course.objects.get(pk=1)

        return HttpResponse(f"Course Id 1 found, data not l
oaded: {c}")

    except Course.DoesNotExist:

        c = Course(
            courseCode="IS2108",
            courseName="Full-stack Software Engineering for
AI Solutions I",
        )
        c.save()
        c = Course(
            courseCode="IS2109", courseName="AI and Machine
Learning Techniques I"
        )
        c.save()
        c = Course(
            courseCode="IS3108",
            courseName="Full-stack Software Engineering for
AI Solutions II",
        )
        c.save()
        c = Course(
            courseCode="IS3109", courseName="AI and Machine
Learning Techniques II"
```

```
        )
        c.save()
        c = Course(courseCode="IS4108", courseName="AI Solu
tioning Capstone Project")
        c.save()
        c = Course(courseCode="IS4151", courseName="AIoT So
lutions and Development")
        c.save()
        c = Course(courseCode="IS4400", courseName="Human-A
I Interaction")
        c.save()
        c = Course(
            courseCode="IS4401", courseName="Generative AI
and Business Applications"
        )
        c.save()
        c = Course(
            courseCode="IS4402", courseName="Machine Learni
ng with Unstructured Data"
        )
        c.save()

        return HttpResponse("Course Id 1 NOT found, data lo
aded successfully.")
```

Add urls.py in app level and edit:

```
from django.urls import path

from . import views

urlpatterns = [
    path("loadData", views.loadData, name="loadData"),
]
```

python manage.py runserver

SETUP for HTML and URLS:

base.html

```
<!-- Base (Master) Templa
te for entire web applica
tion -->

{% load static %}

<!DOCTYPE html>
<html lang="en">

    <head>
        <title>Student Ma
nagement System</title>
        <link rel="styles
heet" type="text/css" hre
f="{% static 'css/styles.
css' %}">
    </head>

    <body>

        <table width="10
0%" width="100%">
            <tr width="10
0%">
                <td colsp
an="2">
                    <head
er>
                        <
h1>Welcome to the Student
Management System</h1>
                    </hea
der>
                </td>
            </tr>
```

```
from django.contrib impor
t admin
from django.urls import p
ath
from . import views



urlpatterns = [
    path('', views.index,
name='adminmodule_inde
x'),
    path('loadData', view
s.loadData, name='loadDat
a'),
    path('createStudent',
views.createStudent, name
='adminmodule_createStude
nt'),
    path('viewAllStudent
s', views.viewAllStudent
s, name='adminmodule_view
AllStudents'),
    path('viewAllCourseRe
gistrations', views.viewA
llCourseRegistrations, na
me='adminmodule_viewAllCo
urseRegistrations'),
    path('registerStudent
ForCourses/<int:studentId
>', views.registerStudent
ForCourses, name='adminmo
dule_registerStudentForCo
urses'),
    path('enterCourseGrad
es/<int:studentId>', view
```

```
            <tr>
            # Side bar
                <td class
="navbar">
                    <nav>
                        <
h2>Menu</h2>
                        <
ul>

<li><a href="{% url 'stud
entmgtsys_index' %}">Home
</a></li>

<li>

# from app level urls.py
name='...'

<a href="{% url 'adminmod
ule_index' %}">Admin Modu
le</a>

<ul>

<li><a href="{% url 'admi
nmodule_createStudent'
%}">Create Student</a></l
i>

<li><a href="{% url 'admi
nmodule_viewAllStudents'
%}">View All Students</a>
</li>

<li><a href="{% url 'admi
nmodule_viewAllCourseRegi
strations' %}">View All C
```

```
s.enterCourseGradesForStu
dent, name='adminmodule_e
nterCourseGrades'),
]
```

```
{% extends "base.html" %}

{% block content %}
    <h2>Student Managemen
t System :: Admin Module
:: View All Students</h2>
    <table border="1" wid
th="100%">
        <tr>
            <th>Student I
D</th>
            <th>Student N
umber</th>
            <th>Full Name
</th>
            <th>Action</t
h>
        </tr>

        {% for student in
students %}
            <tr>
                <td>{{ st
udent.studentId }}</td>
                <td>{{ st
udent.studentNumber }}</t
d>
                <td>{{ st
udent.fullName }}</td>
                <td>
                    <a hr
ef="{% url 'adminmodule_r
egisterStudentForCourses'
```

```
ourse Registrations</a></li>

</ul>

</li>

<li>

<a href="{% url 'studentmodule_index' %}">Student Module</a>

<ul>

<li><a href="{% url 'studentmodule_viewCourseResults' %}">View Course Results</a></li>

</ul>

</li>

</ul>
                    </nav>
                </td>
                # Main content
                <td class="maincontent">
                    <!-- Main content area -->
                    <main>

{% block content %}
```

```
student.studentId %}">Register Student for Courses</a> |
                    <a href="{% url 'adminmodule_enterCourseGrades' student.studentId %}">Enter Course Grades</a>
                </td>
            </tr>
        {% empty %}
            <tr>
                <td colspan="3" style="text-align: center;">No students found.</td>
            </tr>
        {% endfor %}
    </table>
{% endblock %}
```

```
{% endblock %}
                    </mai
n>
                </td>
            </tr>

            <tr width="10
0%">
                <td colsp
an="2">
                    <foot
er>
                        <
hr/
                        <
p>&copy; 2026 IS2108 Full
-stack Software Engineeri
ng for AI Solutions</p>
                    </foo
ter>
                </td>
            </tr>

        </table>

    </body>

</html>
```

```
from django.shortcuts import render
from django.http import HttpResponse
from django.db.models import Prefetch

from .models import Course, Student, CourseGrade
from .forms import StudentForm
```

```
# Create your views here.

def index(request):

    return render(request, 'adminmodule/index.html')



def loadData(request):

    try:

        c = Course.objects.get(pk=1)

        return HttpResponse(f'Course Id 1 found, data not l
oaded: {c}')

    except Course.DoesNotExist:

        c = Course(courseCode='IS2108', courseName='Full-st
ack Software Engineering for AI Solutions I')
        c.save()
        c = Course(courseCode='IS2109', courseName='AI and
Machine Learning Techniques I')
        c.save()
        c = Course(courseCode='IS3108', courseName='Full-st
ack Software Engineering for AI Solutions II')
        c.save()
        c = Course(courseCode='IS3109', courseName='AI and
Machine Learning Techniques II')
        c.save()
        c = Course(courseCode='IS4108', courseName='AI Solu
tioning Capstone Project')
        c.save()
        c = Course(courseCode='IS4151', courseName='AIoT So
lutions and Development')
```

```
        c.save()
        c = Course(courseCode='IS4400', courseName='Human-A
I Interaction')
        c.save()
        c = Course(courseCode='IS4401', courseName='Generat
ive AI and Business Applications')
        c.save()
        c = Course(courseCode='IS4402', courseName='Machine
Learning with Unstructured Data')
        c.save()

        return HttpResponse('Course Id 1 NOT found, data lo
aded successfully.')


# Create User with form
def createStudent(request):

    newForm = StudentForm().render('adminmodule/form_templa
te.html')

    if request.method == 'GET':

        return render(request, 'adminmodule/createStudent.h
tml', {'form': newForm})

    elif request.method == 'POST':

        form = StudentForm(request.POST)

        if form.is_valid():

            cleaned_data = form.cleaned_data

            s = Student(
                    studentNumber=cleaned_data['studentNumb
er'],
                    fullName=cleaned_data['fullName']
```

```
                )
            s.save()

            return render(request, 'adminmodule/createStude
nt.html', {'form': newForm, 'success_message': f'Student
{s.studentId} created successfully.'})

        else:

            return render(request, 'adminmodule/createStude
nt.html', {'form': newForm, 'error_message': 'Form is inval
id. Please correct the errors below.', 'form_errors': form.
errors})

# view all detail
def viewAllStudents(request):

    students = Student.objects.all()

    return render(request, 'adminmodule/viewAllStudents.htm
l', {'students': students})


# create items for existing object
def registerStudentForCourses(request, studentId):

    if request.method == 'GET':

        student = Student.objects.get(pk=studentId)
        courses = Course.objects.all()

        return render(request, 'adminmodule/registerStudent
ForCourses.html', {'student': student, 'courses': courses})

    elif request.method == 'POST':

        student = Student.objects.get(pk=studentId)
```

```
        keys = request.POST.keys()

        for courseId in keys:

            if courseId.startswith('course_'):

                coursePk = int(courseId.split('_')[1])
                course = Course.objects.get(pk=coursePk)
                student.courses.add(course)

        student.save()

        return render(request, 'adminmodule/registerStudent
ForCourses.html', {'student': student, 'success_message':
f'Student {student.studentId} registered for courses succes
sfully.'})


# same as above
def enterCourseGradesForStudent(request, studentId):

    if request.method == 'GET':

        student = Student.objects.get(pk=studentId)
        coursesRegisteredToStudent = student.courses.all()

        return render(request, 'adminmodule/enterCourseGrad
eForStudent.html', {'student': student, 'coursesRegisteredT
oStudent': coursesRegisteredToStudent})

    elif request.method == 'POST':

        student = Student.objects.get(pk=studentId)

        keys = request.POST.keys()

        for courseId in keys:
```

```
            if courseId.startswith('grade_'):

                coursePk = int(courseId.split('_')[1])
                course = Course.objects.get(pk=coursePk)

                courseGrade = CourseGrade(
                    finalLetterGrade=request.POST[courseI
d],
                    student=student,
                    course=course
                )
                courseGrade.save()

        return render(request, 'adminmodule/enterCourseGrad
eForStudent.html', {'student': student, 'success_message':
f'Course grades for Student {student.studentId} entered suc
cessfully.'})



def viewAllCourseRegistrations(request):

    courseRegistrations = Student.objects.prefetch_related(
        Prefetch('courses',
                 queryset=Course.objects.order_by('courseI
d')
        )
    ).all()

    return render(request, 'adminmodule/viewAllCourseRegist
rations.html', {'courseRegistrations': courseRegistration
s})
```

FORM: Create a user using a form with Error Handling

form.html

form_template.html

```
from django import forms

from .models import Stude
nt



class StudentForm(forms.M
odelForm):

    class Meta:

        model = Student
        fields = ['studen
tNumber', 'fullName']
```

views.py

```
# Create User with form
def createStudent(reques
t):

    newForm = StudentForm
().render('adminmodule/fo
rm_template.html')

    if request.method ==
'GET':

        return render(req
uest, 'adminmodule/create
Student.html', {'form': n
ewForm})

    elif request.method =
= 'POST':

        form = StudentFor
```

```
{% for field in form %}

    <tr>
        <td width="20%"><
b><label for="{{ field.id
_for_label }}">{{ field.l
abel_tag }}</label></b></
td>
        <td width="60%">
{{ field }}</td>
        <td style="color:
red;">{{ field.errors }}
</td>
    </tr>

{% endfor %}
```

createStudent.html

```
{% extends "base.html" %}

{% block content %}
    <h2>Student Managemen
t System :: Admin Module
:: Create Student</h2>

    {% if error_message
%}
        <p style="color:
red;">
            {{ error_mess
age }}

            {{ form_error
s }}
        </p>
    {% endif %}
```

```
m(request.POST)

        if form.is_valid
():

            cleaned_data
= form.cleaned_data

            s = Student(
                    stude
ntNumber=cleaned_data['st
udentNumber'],
                    fullN
ame=cleaned_data['fullNam
e']
                )
            s.save()

            return render
(request, 'adminmodule/cr
eateStudent.html', {'for
m': newForm, 'success_mes
sage': f'Student {s.stude
ntId} created successfull
y.'})

        else:

            return render
(request, 'adminmodule/cr
eateStudent.html', {'for
m': newForm, 'error_messa
ge': 'Form is invalid. Pl
ease correct the errors b
elow.', 'form_errors': fo
rm.errors})
```

```
    {% if success_message
%}
        <p style="color:
blue;">{{ success_message
}}</p>
    {% endif %}



    <form method="post">
        {% csrf_token %}

        <table border="1"
width="100%">
            {{ form }}

            <tr>
                <td colsp
an="3" style="text-align:
center;">
                    <inpu
t type="reset" value="Cle
ar">
                    <inpu
t type="submit" value="Cr
eate Student">
                </td>
            </tr>

        </table>
    </form>
{% endblock %}
```

View All Students and able to direct to individual info

```
# view all detail
def viewAllStudents(reque
st):

    students = Student.ob
jects.all()

    return render(reques
t, 'adminmodule/viewAllSt
udents.html', {'student
s': students})
```

```
{% extends "base.html" %}

{% block content %}
    <h2>Student Managemen
t System :: Admin Module
:: View All Students</h2>
    <table border="1" wid
th="100%">
        <tr>
            <th>Student I
D</th>
            <th>Student N
umber</th>
            <th>Full Name
</th>
            <th>Action</t
h>
        </tr>

        {% for student in
students %}
            <tr>
                <td>{{ st
udent.studentId }}</td>
                <td>{{ st
udent.studentNumber }}</t
d>
                <td>{{ st
```

View the Individual Student Info and Update info with Multiselect

```
udent.fullName }}</td>
                <td>
                    <a hr
ef="{% url 'adminmodule_r
egisterStudentForCourses'
student.studentId %}">Reg
ister Student for Courses
</a> |
                    <a hr
ef="{% url 'adminmodule_e
nterCourseGrades' studen
t.studentId %}">Enter Cou
rse Grades</a>
                </td>
            </tr>
        {% empty %}
            <tr>
                <td colsp
an="3" style="text-align:
center;">No students foun
d.</td>
            </tr>
        {% endfor %}
    </table>
{% endblock %}
```

```
# create items for existi
ng object
def registerStudentForCou
rses(request, studentId):

    if request.method ==
'GET':

        student = Studen
t.objects.get(pk=studentI
d)
        courses = Course.
```

```
 {% extends "base.html"
%}

{% block content %}
    <h2>Student Managemen
t System :: Admin Module
:: Register Student for C
ourses</h2>

    <h3>Student Details</
h3>
```

```
objects.all()

        return render(req
uest, 'adminmodule/regist
erStudentForCourses.htm
l', {'student': student,
'courses': courses})

    elif request.method =
= 'POST':

        student = Studen
t.objects.get(pk=studentI
d)

        keys = request.PO
ST.keys()

        for courseId in k
eys:

            if courseId.s
tartswith('course_'):

                coursePk
= int(courseId.split('_')
[1])
                course =
Course.objects.get(pk=cou
rsePk)
                student.c
ourses.add(course)

        student.save()

        return render(req
uest, 'adminmodule/regist
erStudentForCourses.htm
```

```
    {% if success_message
%}
        <p style="color:
blue;">{{ success_message
}}</p>
    {% endif %}

    <table border="1" wid
th="50%">

        <tr>
            <th width="3
0%">Student ID</th>
            <td>{{ studen
t.studentId }}</td>
        </tr>
        <tr>
            <th>Student N
umber</th>
            <td>{{ studen
t.studentNumber }}</td>
        </tr>
        <tr>
            <th>Full Name
</th>
            <td>{{ studen
t.fullName }}</td>
        </tr>

    </table>

    {% if not success_mes
sage %}

    <h3>Courses Available
for Registration</h3>

    <form method="post">
```

```
l', {'student': student,
'success_message': f'Stud
ent {student.studentId} r
egistered for courses suc
cessfully.'})
```

```
        {% csrf_token %}

        <table border="1"
width="100%">
            <tr>
                <th>Cours
e Id</th>
                <th>Cours
e Code</th>
                <th>Cours
e Name</th>
                <th>Selec
t</th>
            </tr>

            {% for course
in courses %}
                <tr>
                    <td>
{{ course.courseId }}</td
>
                    <td>
{{ course.courseCode }}</
td>
                    <td>
{{ course.courseName }}</
td>
                    <td s
tyle="text-align: cente
r;">
                        <
input type="checkbox" nam
e="course_{{ course.cours
eId }}">
                    </td>
                </tr>
            {% endfor %}
```

```
            <tr>
                <td colsp
an="4" style="text-align:
center;">
                    <inpu
t type="submit" value="Re
gister Selected Courses">
                </td>
            </tr>
        </table>
    </form>

    {% endif %}

{% endblock %}
```

Enter Course Grade (Update)

```
def enterCourseGradesForS
tudent(request, studentI
d):

    if request.method ==
'GET':

        student = Studen
t.objects.get(pk=studentI
d)
        coursesRegistered
ToStudent = student.cours
es.all()

        return render(req
uest, 'adminmodule/enterC
ourseGradeForStudent.htm
l', {'student': student,
'coursesRegisteredToStude
```

```
{% extends "base.html" %}

{% block content %}
    <h2>Student Managemen
t System :: Admin Module
:: Enter Course Grades fo
r Student</h2>

    <h3>Student Details</
h3>

    {% if success_message
%}
        <p style="color:
blue;">{{ success_message
}}</p>
    {% endif %}

    <table border="1" wid
```

```
nt': coursesRegisteredToStudent})

    elif request.method == 'POST':

        student = Student.objects.get(pk=studentId)

        keys = request.POST.keys()

        for courseId in keys:

            if courseId.startswith('grade_'):

                coursePk = int(courseId.split('_')[1])
                course = Course.objects.get(pk=coursePk)

                courseGrade = CourseGrade(
                    finalLetterGrade=request.POST[courseId],
                    student=student,
                    course=course
                )
                courseGrade.save()
```

```
th="50%">

        <tr>
            <th width="30%">Student ID</th>
            <td>{{ student.studentId }}</td>
        </tr>
        <tr>
            <th>Student Number</th>
            <td>{{ student.studentNumber }}</td>
        </tr>
        <tr>
            <th>Full Name</th>
            <td>{{ student.fullName }}</td>
        </tr>

    </table>

    {% if not success_message %}

    <h3>Courses Registered to Student</h3>

    <form method="post">
        {% csrf_token %}

        <table border="1" width="100%">
            <tr>
                <th>Course Id</th>
                <th>Cours
```

```
        return render(request, 'adminmodule/enterCourseGradeForStudent.html', {'student': student, 'success_message': f'Course grades for Student {student.studentId} entered successfully.'})
```

```
e Code</th>
                <th>Course Name</th>
                <th>Select</th>
            </tr>

            {% for course in coursesRegisteredToStudent %}
                <tr>
                    <td>{{ course.courseId }}</td>
                    <td>{{ course.courseCode }}</td>
                    <td>{{ course.courseName }}</td>
                    <td style="text-align: center;">
                        <select name="grade_{{ course.courseId }}">

<option value="">-- Select Grade --</option>

<option value="A+">A+</option>

<option value="A">A</option>

<option value="A-">A-</option>
```

```
<option value="B+">B+</option>

<option value="B">B</option>

<option value="C+">C+</option>

<option value="C">C</option>

<option value="D+">D+</option>

<option value="D">D</option>

<option value="E">E</option>

<option value="F">F</option>

</select>
                    </td>
                </tr>
            {% endfor %}

            <tr>
                <td colspan="4" style="text-align: center;">
                    <input type="submit" value="Register Selected Courses">
                </td>
```

```
            </tr>
        </table>
    </form>

    {% endif %}

{% endblock %}
```

View All Course Regist ion

```
def viewAllCourseRegistrations(request):

    courseRegistrations = Student.objects.prefetch_related(
        Prefetch('courses',
                 queryset=Course.objects.order_by('courseId')
        )
    ).all()

    return render(request, 'adminmodule/viewAllCourseRegistrations.html', {'courseRegistrations': courseRegistrations})
```

```
{% extends "base.html" %}

{% block content %}
    <h2>Student Management System :: Admin Module :: View All Course Registrations</h2>
    <table border="1" width="100%">
        <tr>
            <th>Student Number</th>
            <th>Student Full Name</th>
            <th>Course Code</th>
            <th>Course Name</th>
        </tr>

        {% for student in courseRegistrations %}

            {% for course in student.courses.all %}

            <tr>
```

```
                <td>{{ st
udent.studentNumber }}</t
d>
                <td>{{ st
udent.fullName }}</td>
                <td>{{ co
urse.courseCode }}</td>
                <td>{{ co
urse.courseName }}</td>
            </tr>

            {% endfor %}
        {% empty %}
            <tr>
                <td colsp
an="4" style="text-align:
center;">No course regist
rations found.</td>
            </tr>
        {% endfor %}
    </table>
{% endblock %}
```

Student View Course Result

```
from django.shortcuts imp
ort render
from adminmodule.models i
mport Student, CourseGrad
e



# Create your views here.

def index(request):
```

```
{% extends "base.html" %}

{% block content %}
    <h2>Student Managemen
t System :: Student Modul
e :: View Course Results
</h2>

    <h3>Student Details</
h3>

    <form method="post">
```

```
    return render(reques
t, 'studentmodule/index.h
tml')



def viewCourseResults(req
uest):

    if request.method ==
'GET':

        return render(req
uest, 'studentmodule/view
CourseResults.html')

    elif request.method =
= 'POST':

        student = Studen
t.objects.get(studentNumb
er=request.POST.get('stud
entNumber'))
        courseGrades = Co
urseGrade.objects.filter
(student=student).prefetc
h_related('course')

        return render(req
uest, 'studentmodule/view
CourseResults.html', {'st
udent': student, 'courseG
rades': courseGrades})
```

```
        {% csrf_token %}

        <table border="1"
width="50%">
            <tr>
                <th>Enter
Student Number: </th>
                <td>
                    <inpu
t type="text" name="stude
ntNumber" required>
                </td>
            </tr>
            <tr>
                <td colsp
an="2" style="text-align:
center;">
                    <inpu
t type="submit" value="Vi
ew Course Results">
                </td>
            </tr>
        </table>
    </form>

    {% if courseGrades %}

    <h3>Course Results</h
3>

    <table border="1" wid
th="100%">
        <tr>
            <th>Course Co
de</th>
            <th>Course Na
me</th>
            <th>finalLett
```

```
erGrade</th>
        </tr>

        {% for courseGrad
e in courseGrades %}

            <tr>
                <td>{{ co
urseGrade.course.courseCo
de }}</td>
                <td>{{ co
urseGrade.course.courseNa
me }}</td>
                <td>{{ co
urseGrade.finalLetterGrad
e }}</td>
            </tr>

        {% empty %}
            <tr>
                <td colsp
an="4" style="text-align:
center;">No course result
s found.</td>
            </tr>
        {% endfor %}
    </table>

    {% endif %}

{% endblock %}
```